18 DE DICIEMBRE DE 2024

PRÁCTICA OPEN MP Y CUDA

JOKIN IZAGUIRRE PÉREZ Y AROA GÓMEZ RETA
SISTEMAS PARALELOS Y DISTRIBUIDOS
2º GIA

ÍNDICE

1. **Introducción**

2. **Ejercicio 1: integral**
    1. **Versión en serie**
    2. **Versión Open MP**
    3. **Versión Cuda**

3. **Ejercicio 2: mxv**
    1. **Versión en serie**
    2. **Versión Open MP**
    3. **Versión Cuda**
    4. **Comparación de modelos**

4. **Conclusiones**

Introducción

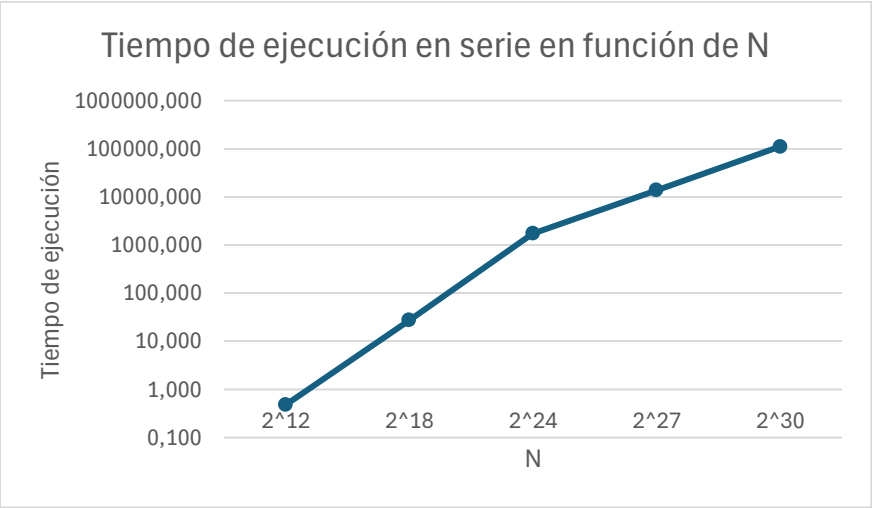
En este trabajo se van a realizar dos ejercicios que consisten en paralelizar programas informáticos de dos formas diferentes: mediante Open Mp y utilizando el kernel de la GPU, para conseguir un mayor rendimiento. Seguidamente se van a medir sus tiempos de ejecución en serie, con Open MP y en el kernel. Así mismo, se va a calcular su factor de aceleración. Esto se va a representar en diferentes tablas y gráficos para su interpretación y para así poder ver cuál es la opción con un mejor rendimiento.

Ejercicio 1: integral

1. Versión en serie:

Para la versión en serie del programa integral, que calcula la integral de un valor mediante la suma del área de N trapecios bajo la curva de la función (a mayor N mayor precisión), hemos obtenido los tiempos de ejecución para algunos valores de N y los hemos representado en una tabla y un gráfico que se muestran a continuación. Como podemos observar en ellos, el programa es eficiente para los valores más pequeños de N (para los que la integral es menos exacta), pero tiene un gran coste para valores grandes de N (mejores para el cálculo).

TIEMPO DE EJECUCIÓN (ms)	
N	En serie
2^12	0,478
2^18	27,343
2^24	1731,948
2^27	13855,197
2^30	111524,722



2. Versión Open MP:

```
double Integrar (double a, double b, int n, double w)
{
  double  resultado, x;
  int     i;

  resultado = (f(a) + f(b)) / 2.0;

  #pragma omp parallel for private(i,x) shared(a,w), reduction(+:resultado) schedule(static)
  for (i=1; i<n; i++)
  {
    x = a + i*w;
    resultado += f(x);
  }
  resultado *=  w;

  return (resultado);
}
```

En este caso, la parte paralelizable del programa es el bucle de la función integrar. Para ello establecemos las variables i y x como privadas, ya que cada hilo necesita tener las suyas propias. Declaramos a y w como compartidas ya que todos los hilos las utilizan en sus operaciones sin modificarlas. La variable resultado es de reducción, ya que todos los hilos la leen y la reescriben, lo que podría dar problemas si dos hilos acceden a ella a la vez.

El reparto es estático ya que todas las iteraciones tienen un coste similar, ya que realizan las mismas operaciones con diferentes valores.

No podemos introducir un nowait al final del bucle ya que necesitamos tener la variable resultado calculada inmediatamente después del bucle.

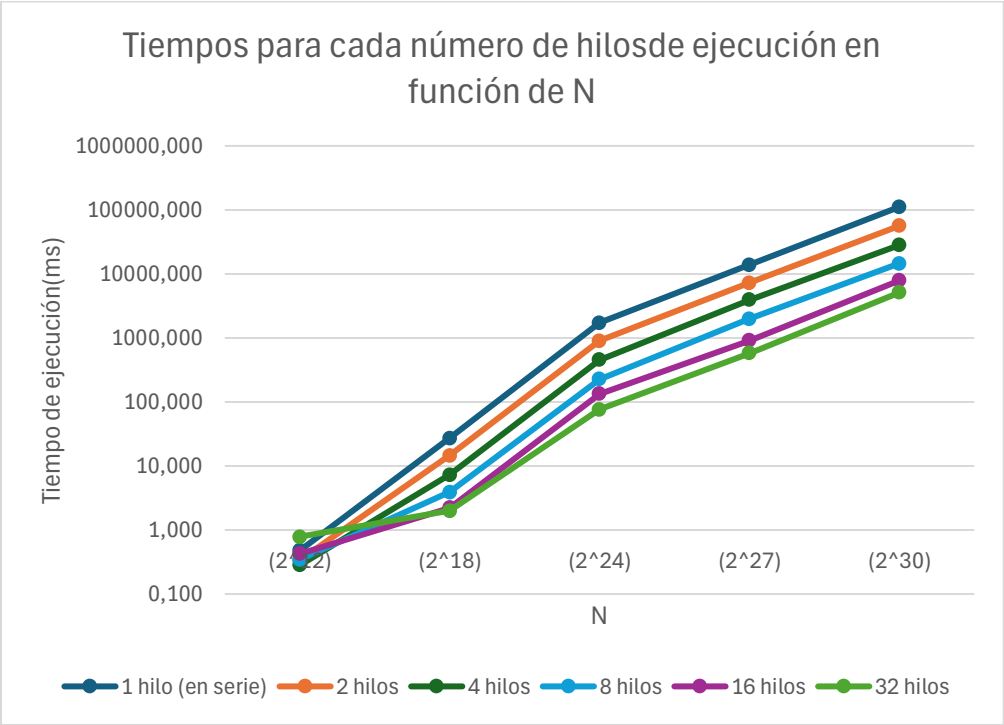
Como podemos observar en las tablas y gráficos a continuación, el factor de aceleración aumenta en función del número de hilos y del número de N, hasta llegar a N = 2^27. En N=2^30 empieza a disminuir el factor de aceleración y, por lo tanto, la eficiencia. Observamos que a mayor número de hilos el programa es más eficiente con N mayores, por lo que podemos decir que conviene paralelizar el programa para N grandes. Con valores pequeños de N y un gran número de hilos puede tardar más en ejecutarse en paralelo que en serie.

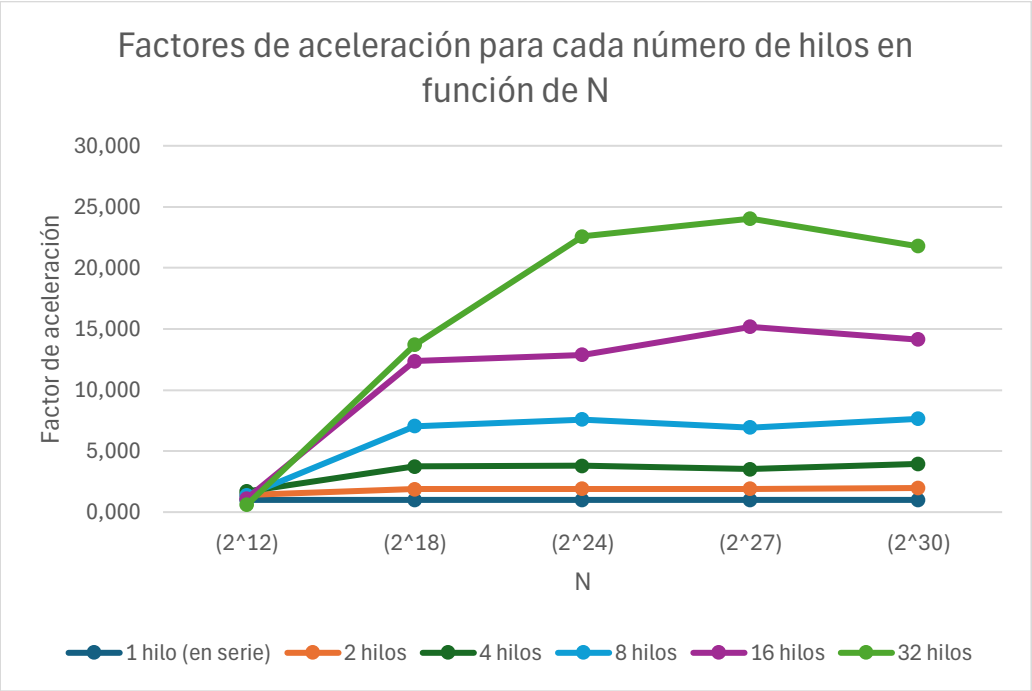
TIEMPO DE EJECUCIÓN (ms)						
N	1 hilo (serie)	2 hilos	4 hilos	8 hilos	16 hilos	32 hilos
2^12	0,478	0,339	0,282	0,344	0,424	0,786
2^18	27,343	14,643	7,298	3,879	2,213	1,997
2^24	1731,948	904,352	456,786	228,079	134,339	76,734

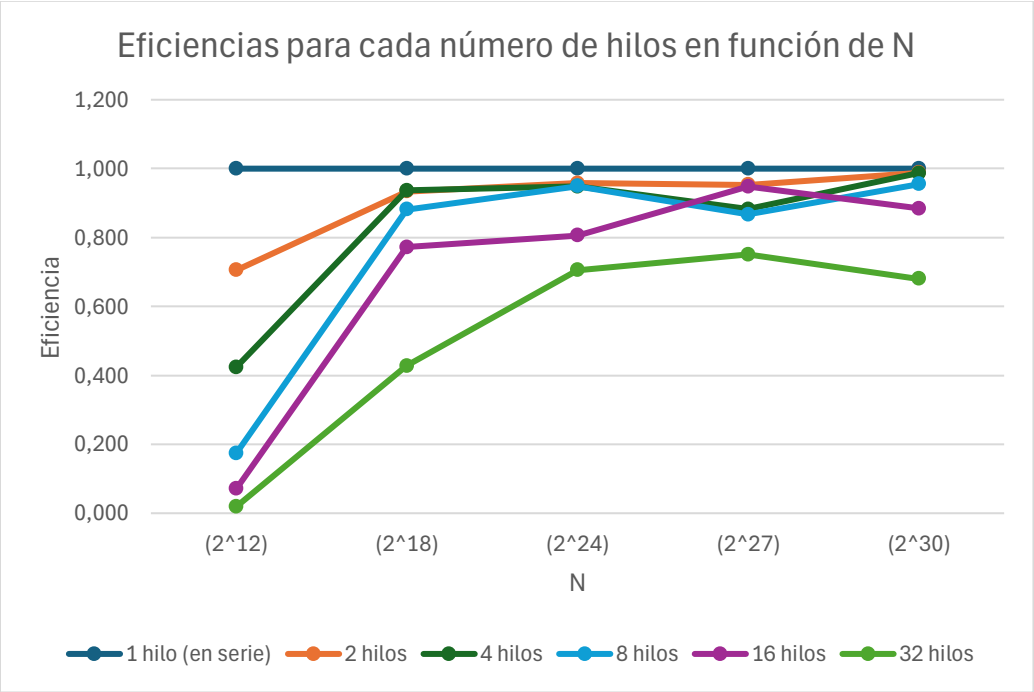
2^27	13855,197	7269,295	3923,687	1997,755	913,328	576,460
2^30	111524,722	56460,001	28238,273	14601,698	7883,979	5119,661

FACTOR DE ACELERACIÓN						
N	1 hilo (serie)	2 hilos	4 hilos	8 hilos	16 hilos	32 hilos
2^12	1,000	1,410	1,695	1,390	1,127	0,608
2^18	1,000	1,867	3,747	7,049	12,356	13,692
2^24	1,000	1,915	3,792	7,594	12,892	22,571
2^27	1,000	1,906	3,531	6,935	15,170	24,035
2^30	1,000	1,975	3,949	7,638	14,146	21,784

EFICIENCIA						
N	1 hilo (serie)	2 hilos	4 hilos	8 hilos	16 hilos	32 hilos
2^12	1,000	0,705	0,424	0,174	0,070	0,019
2^18	1,000	0,934	0,937	0,881	0,772	0,428
2^24	1,000	0,958	0,948	0,949	0,806	0,705
2^27	1,000	0,953	0,883	0,867	0,948	0,751
2^30	1,000	0,988	0,987	0,955	0,884	0,681




Factores de aceleración para cada número de hilos en función de N
Factor de aceleración
30,000
25,000
20,000
15,000
10,000
5,000
0,000
(2^12)
(2^18)
(2^24)
(2^27)
(2^30)
N
1 hilo (en serie)
2 hilos
4 hilos
8 hilos
16 hilos
32 hilos


Eficiencias para cada número de hilos en función de N
Eficiencia
1,200
1,000
0,800
0,600
0,400
0,200
0,000
(2^12)
(2^18)
(2^24)
(2^27)
(2^30)
N
1 hilo (en serie)
2 hilos
4 hilos
8 hilos
16 hilos
32 hilos

3. Versión Cuda:

```
__device__ double fd (double x)
{
  double  y;

  y = 1.0 / (sin(x) + 2.0) + 1.0 / (sin(x)*cos(x) + 2.0);
  return (y);
}

__global__ void Integrar (double a, double b, int n, double w, double *out)
{
  extern __shared__ double temp[];

  double x, aux;

  int idx = (blockDim.x * blockIdx.x) + threadIdx.x;
  int stride = blockDim.x * gridDim.x;
  int tid = threadIdx.x;

  aux = 0.0;
  for (int i=idx; i<n; i+=stride) {
    if (i > 0) {
    x =  (a + i*w);
    aux += fd(x);
    }
  }

  temp[tid] = aux;
  __syncthreads();

  for (int stride=1; stride<blockDim.x; stride*=2) {
      if (((tid % (2*stride)) == 0) && ((tid+stride) < blockDim.x)) temp[tid] += temp[tid + stride];
      __syncthreads();
  }

  if (tid == 0) out[blockIdx.x] = temp[tid];
}
```

En este caso, la función a ejecutar en en Kernel es la función Integrar. Como esta hace uso de la función f, tenemos que definir f en el kernel. En este caso la llamaremos fd, ya que también necesitamos utilizar la función f en la CPU, por lo que tendremos que definirla también en el host (fh).

En este caso, hemos pasado por parámetro el valor de a, b, n,w y la dirección de un vector de salida out. Para la ejecución de la función, en cada bloque definimos un vector temporal dinámico con el tamaño de la dimensión del bloque, que se le pasa al llamar el kernel desde la GPU. Este vector lo utilizaremos para realizar la suma de sus elementos utilizando la estrategia de reducción, para acabar almacenando toda la suma en la posición 0.

Como la función sirve para cualquier número y tamaño de bloque, tenemos que asegurarnos de que se procesan todos los elementos (n). Para ello, puede que cada hilo, es decir, cada posición del vector temp, tenga que procesar más de un elemento. Esto lo conseguimos con el primer for, en el que almacenamos en aux la suma de aplicar la función a los elementos que correspondan a cada hilo, y después lo guardamos en la posición correspondiente. Después de esto sincronizamos todos los hilos para asegurar que todas las posiciones del vector tengan su valor antes de comenzar las sumas logarítmicas.

En el segundo for, sumamos de manera logarítmica todos los elementos del vector hasta almacenar el resultado total del bloque en la posición 0. Así evitamos dependencias de lectura y escritura en la suma.

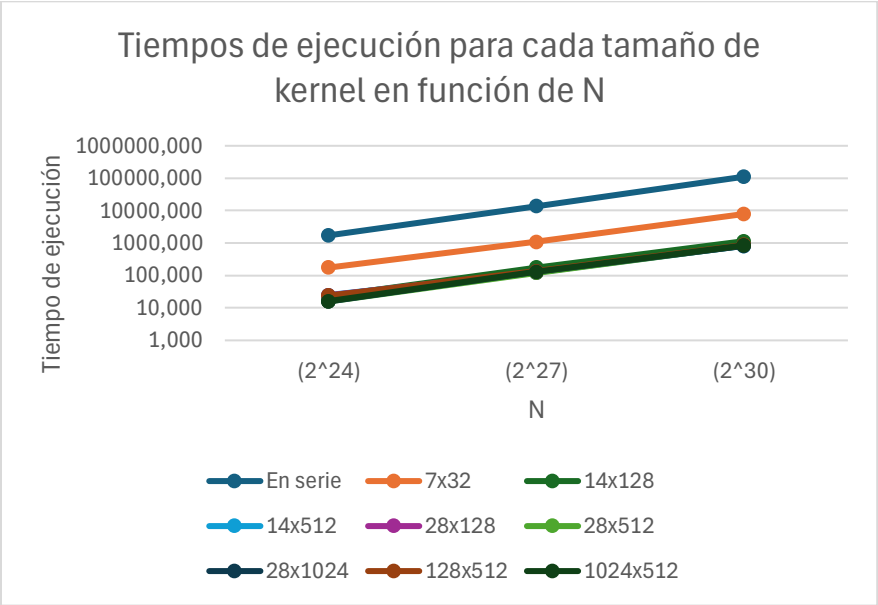
Tras esta operación, tendremos la suma de todos los elementos del bloque en la posición 0 del vector temporal, por lo que la almacenamos en la posición del identificador de bloque del vector out (que tendrá tantas posiciones como número de bloques utilicemos).

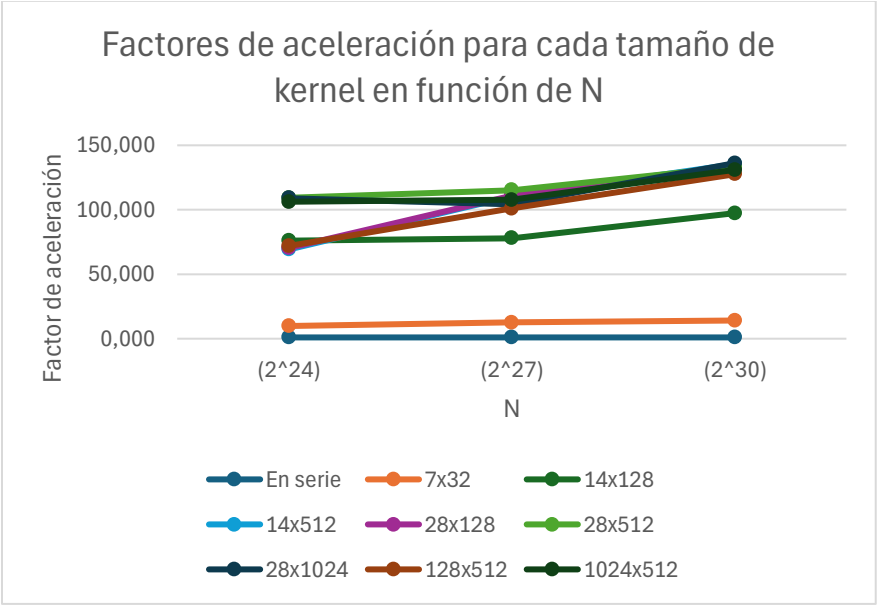
Por último, en la CPU, realizaremos la suma de todos los elementos del vector out en un for desde 0 hasta el número de bloques, para obtener la suma total, que se la tendremos que sumar a (fh(a) + fh(b))/2, calculado en el host.

Como podemos observar en las tablas y gráficas a continuación, el rendimiento del programa mejora notablemente al paralelizarlo con Cuda. El peor de los casos sería ejecutarlo en serie, seguido de la ejecución en en kernel para 7 bloques de 32 hilos, pero para el resto de tamaños del kernel los tiempos son parecidos. El caso más eficiente sería para 28 bloques de 512 hilos, por lo que sería la configuración que elegiríamos a la hora de guardar una configuración definitiva del kernel para este programa.

TIEMPO DE EJECUCIÓN (ms)									
N	1x1 (serie)	7x32	14x128	14x512	28x128	28x512	28x1024	128x512	1024x512
2^24	1731,948	175,702	22,773	25,029	24,641	15,842	15,878	24,082	16,304
2^27	13855,197	1100,774	177,664	125,356	124,835	120,359	132,901	137,070	128,597
2^30	111524,722	7900,577	1145,167	823,615	843,222	836,382	819,064	874,173	851,450

FACTOR DE ACELERACIÓN									
N	1x1 (serie)	7x32	14x128	14x512	28x128	28x512	28x1024	128x512	1024x512
2^24	1,000	9,857	76,053	69,198	70,287	109,326	109,078	71,919	106,228
2^27	1,000	12,587	77,985	110,527	110,988	115,116	104,252	101,081	107,741
2^30	1,000	14,116	97,387	135,409	132,260	133,342	136,161	127,577	130,982


Tiempos de ejecución para cada tamaño de kernel en función de N
Tiempo de ejecución
1000000,000
100000,000
10000,000
1000,000
100,000
10,000
1,000
(2^24)
(2^27)
(2^30)
N
En serie
7x32
14x128
14x512
28x128
28x512
28x1024
128x512
1024x512

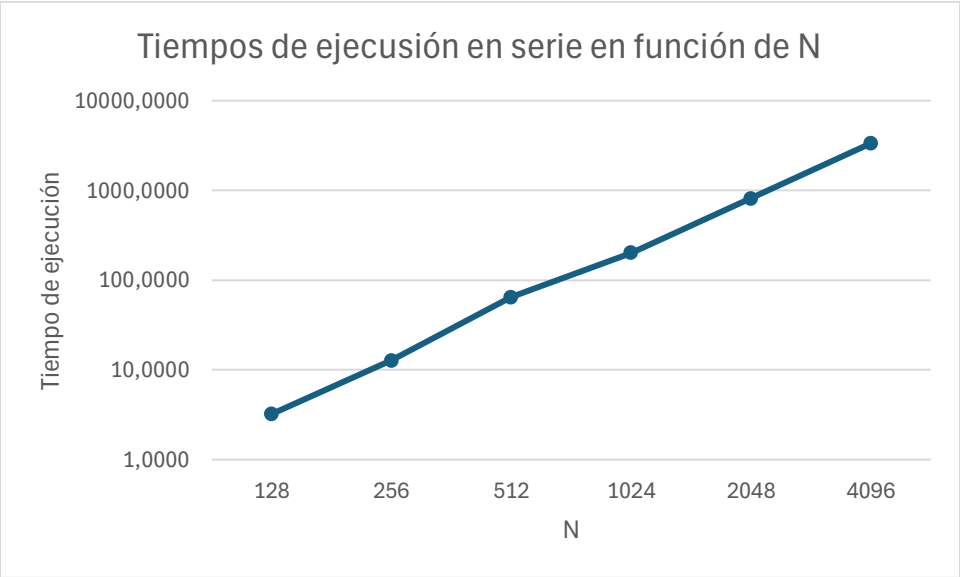

Factores de aceleración para cada tamaño de kernel en función de N
Factor de aceleración
150,000
100,000
50,000
0,000
(2^24)
(2^27)
(2^30)
N
En serie
7x32
14x128
14x512
28x128
28x512
28x1024
128x512
1024x512

Ejercicio 2: mxv

1. Versión en serie:

Para la versión en serie del programa mxv, que lee un fichero (fmat.txt) y escribe los resultados de los productos obtenidos en todos los experimentos en los que cada matriz se procesa 100 veces para simular el proceso que se realiza cuando se utilizan redes neuonales, hemos obtenido los tiempos de ejecución (suma de los tiempos de ejecución de cada uno de los experimentos, teniendo en cuenta las 100 repeticiones) en serie. Hemos representado estos tiempos en la tabla y gráfica que se muestran a continuación. Como podemos observar, realizar el programa en serie tiene un coste elevado cuando tratamos valores grandes de N (como se tratan en las redes neuronales).

TIEMPO DE EJECUCIÓN (ms)	
N	En serie
128	3,2193
256	12,7166
512	64,4296
1024	202,5349
2048	812,7538
4096	3346,8291



2. Versión Open MP:

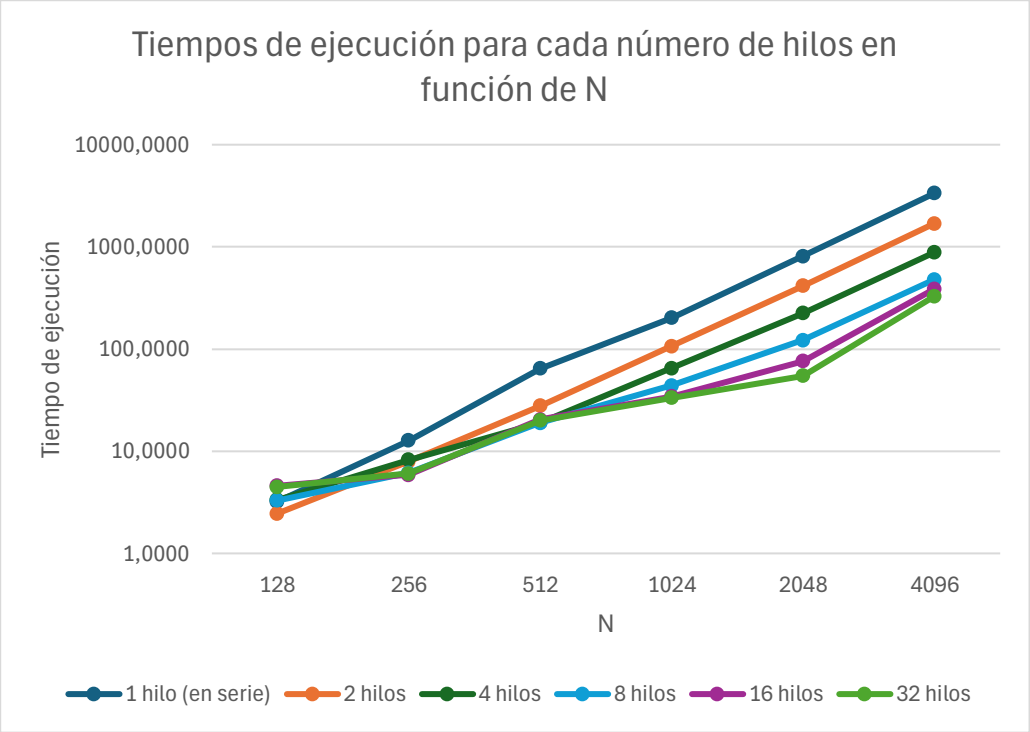
```
void mxv (float *m, float *v, float *p, int N)
{
  int  i, j;

  #pragma omp parallel for private(i,j), shared (m, N, v, p) schedule(dynamic)
  for (i=0; i<N; i++)
  {
    p[i] = 0.0;
    for (j=0; j<N; j++) p[i] += m[i*N+j] * v[j];
  }
}
```

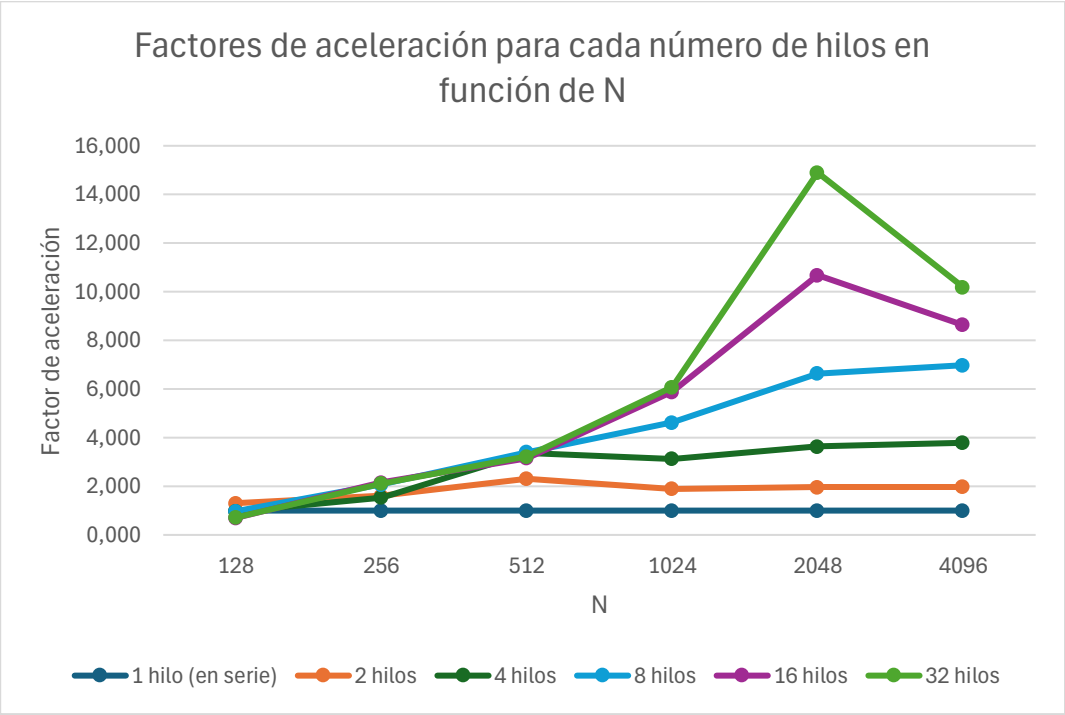
En este caso, la parte paralelizable del programa es el bucle de la función mvx. En este caso establecemos las variables i y j como privadas, ya que son las que se utilizan para iterar en el bucle. Declaramos m, N, v y p como compartidas ya que todos los hilos las utilizan en sus operaciones, y p es el vector del que cada hilo modifica las posiciones correspondientes según el valor de su i privada.
El reparto es dinámico ya que las iteraciones pueden tener un coste diferente dependiendo de los valores de los vectores m y v a los que accedan.
No podemos introducir un nowait al final del bucle ya que en el programa principal necesitamos tener el vector p calculado inmediatamente después del bucle.
Como podemos observar en las tablas y gráficos a continuación, el factor de aceleración aumenta en función del número de hilos y del número de N, hasta llegar a N = 2048. En N=4096 empieza a disminuir el factor de aceleración y, por lo tanto, la eficiencia. Observamos que a mayor número de hilos el programa es más eficiente con N mayores, por lo que podemos decir que conviene paralelizar el programa para N grandes. Con valores pequeños de N y un gran número de hilos puede tardar más en ejecutarse en paralelo que en serie.

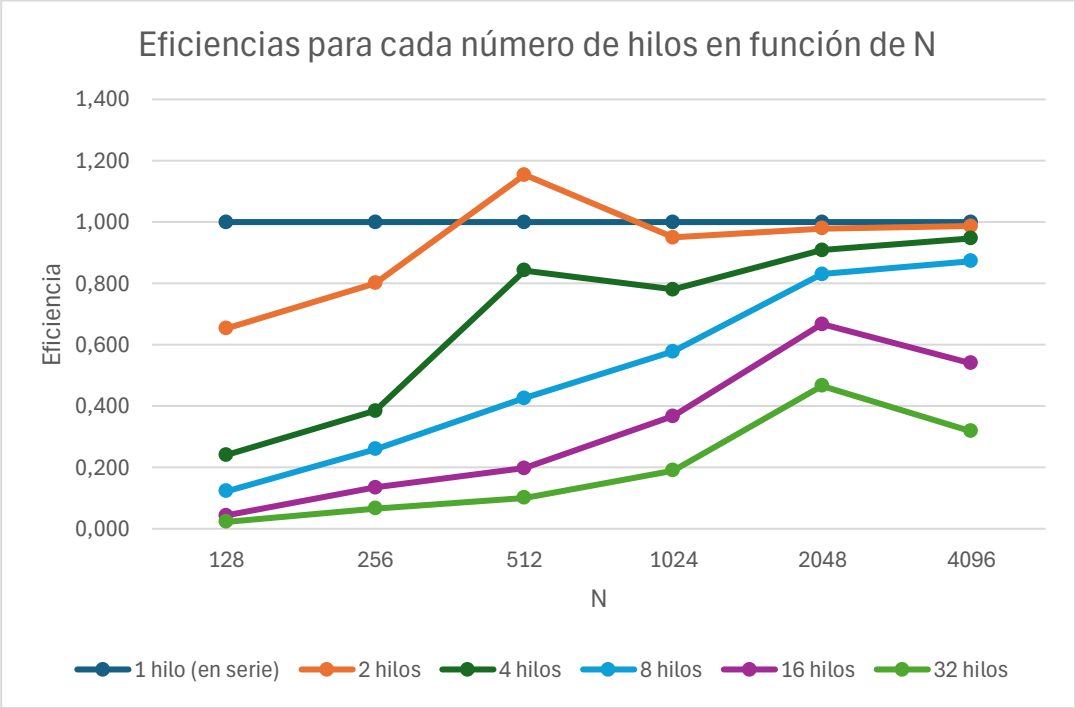
TIEMPO DE EJECUCIÓN (ms)						
N	1 hilo (serie)	2 hilos	4 hilos	8 hilos	16 hilos	32 hilos
128	3,2193	2,4636	3,3422	3,2950	4,5900	4,4786
256	12,7166	7,9346	8,2641	6,1242	5,8830	6,0323
512	64,4296	27,9086	19,1373	18,9246	20,4111	19,9379
1024	202,5349	106,6509	64,8619	43,8240	34,4433	33,2983
2048	812,7538	414,9850	223,7978	122,3709	76,0929	54,5507
4096	3346,8291	1696,6839	883,7386	479,6015	387,4353	328,6763

FACTOR DE ACELERACIÓN						
N	1 hilo (serie)	2 hilos	4 hilos	8 hilos	16 hilos	32 hilos
128	1,000	1,307	0,963	0,977	0,701	0,719
256	1,000	1,603	1,539	2,076	2,162	2,108
512	1,000	2,309	3,367	3,405	3,157	3,232
1024	1,000	1,899	3,123	4,622	5,880	6,082
2048	1,000	1,959	3,632	6,642	10,681	14,899
4096	1,000	1,973	3,787	6,978	8,638	10,183

EFICIENCIA						
N	1 hilo (serie)	2 hilos	4 hilos	8 hilos	16 hilos	32 hilos
128	1,000	0,653	0,241	0,122	0,044	0,022
256	1,000	0,801	0,385	0,260	0,135	0,066
512	1,000	1,154	0,842	0,426	0,197	0,101
1024	1,000	0,950	0,781	0,578	0,368	0,190
2048	1,000	0,979	0,908	0,830	0,668	0,466
4096	1,000	0,986	0,947	0,872	0,540	0,318




Factores de aceleración para cada número de hilos en función de N
Factor de aceleración
16,000
14,000
12,000
10,000
8,000
6,000
4,000
2,000
0,000
128
256
512
1024
2048
4096
N
1 hilo (en serie)
2 hilos
4 hilos
8 hilos
16 hilos
32 hilos


Eficiencias para cada número de hilos en función de N
Eficiencia
1,400
1,200
1,000
0,800
0,600
0,400
0,200
0,000
128
256
512
1024
2048
4096
N
1 hilo (en serie)
2 hilos
4 hilos
8 hilos
16 hilos
32 hilos

3. Versión Cuda:

```
__global__ void mxv (float *m, float *v, float *p, int N)
{
  int i = (blockDim.x * blockIdx.x) + threadIdx.x;
  float suma = 0.0;

  if (i<N){
    for (int j=0; j<N; j++) suma += m[i*N+j] * v[j];
  }

  p[i] = suma;
}
```

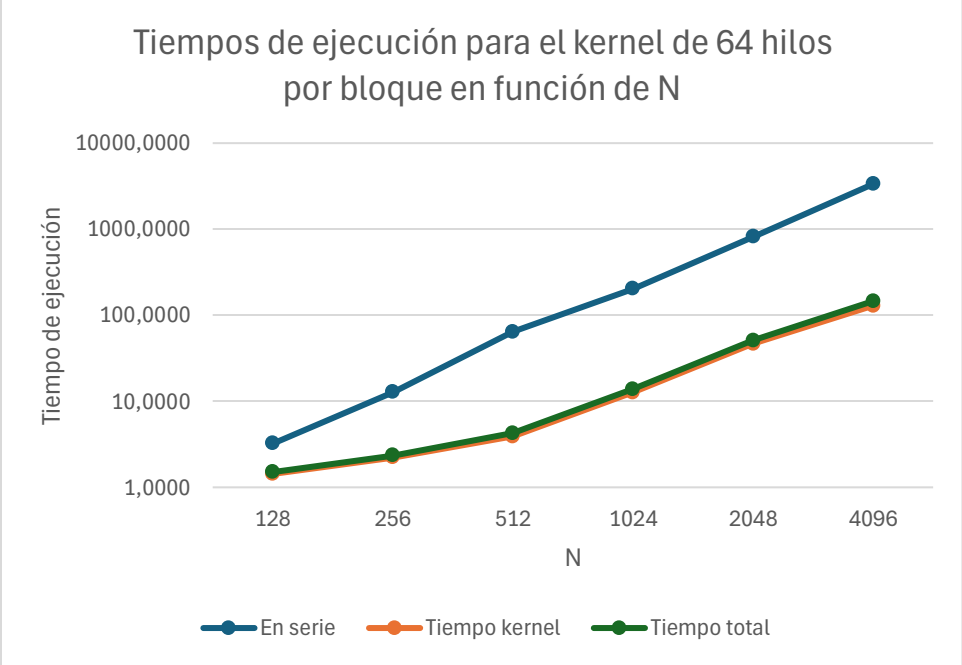
En este caso, la función a ejecutar en en Kernel es la función mxv. Le pasamos por parámetro la dirección de una matriz m, la de un vector v y la de un vector de salida p en el que se almacenará el cálculo del producto. También le pasamos el valor de N, que es la dimensión de los vectores v y p. Así mismo, la matriz m es de dimensiones NxN.

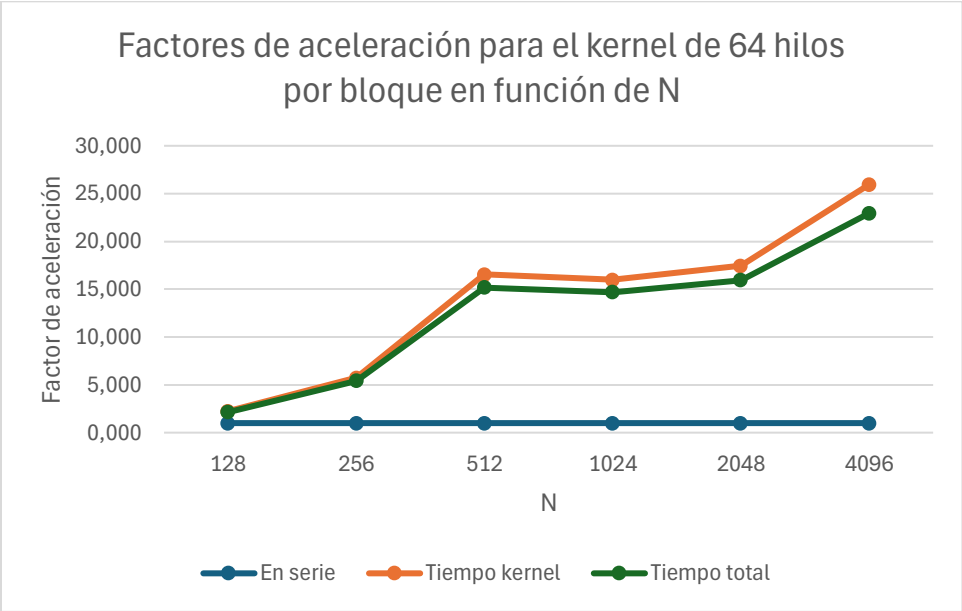
Para está función, elegiremos unas dimensiones del kernel para las que cada hilo ejecute un elemento del vector de salida p. Para ello, antes de llamar a la función en la CPU, pediremos el tamaño de bloque y en función del que sea calcularemos el número de bloques para asegurarnos de que al menos hay tantos hilos como elementos (N). En la función del kernel tenemos que asegurarnos de que se realiza el producto solo para las i que son menores a N, ya que podrían haberse generado más hilos de los necesarios.

Como podemos observar en las tablas y gráficas a continuación, en las que se muestra el tiempo de ejecución en serie y en Cuda con 64 hilos por bloque, teniendo en cuenta el tiempo del kernel y el del kernel y las copias de memoria, el rendimiento del programa mejora notablemente al paralelizarlo con Cuda. Observamos que el tiempo que se tarda en realizar las copias de memoria es muy pequeño, ya que los tiempos de ejecución del kernel son parecidos teniendo en cuenta el coste de las copias o no (a mayor N aumenta, pero compensa respecto al tiempo en serie). Como podemos ver, el programa en Cuda tiene un mayor factor de aceleración conforme N aumenta.

TIEMPO DE EJECUCIÓN (ms)			
N	En serie	Tiempo kernel	Tiempo total
128	3,2193	1,4308	1,5032
256	12,7166	2,2229	2,3429
512	64,4296	3,8983	4,2486
1024	202,5349	12,6640	13,7960
2048	812,7538	46,6134	51,0147
4096	3346,8291	129,0270	145,8942

FACTOR DE ACELERACIÓN			
N	En serie	Tiempo kernel	Tiempo total
128	1,000	2,250	2,142
256	1,000	5,721	5,428
512	1,000	16,528	15,165
1024	1,000	15,993	14,681
2048	1,000	17,436	15,932
4096	1,000	25,939	22,940





4. Comparación de modelos:

Por último, vamos a comparar los tiempos de ejecución y los factores de aceleración del programa mxv en serie, paralelizado con Open MP utilizando 32 hilos y paralelizado con Cuda con 64 hilos por bloque sin tener en cuenta y teniendo en cuenta el tiempo correspondiente a las copias de memoria.

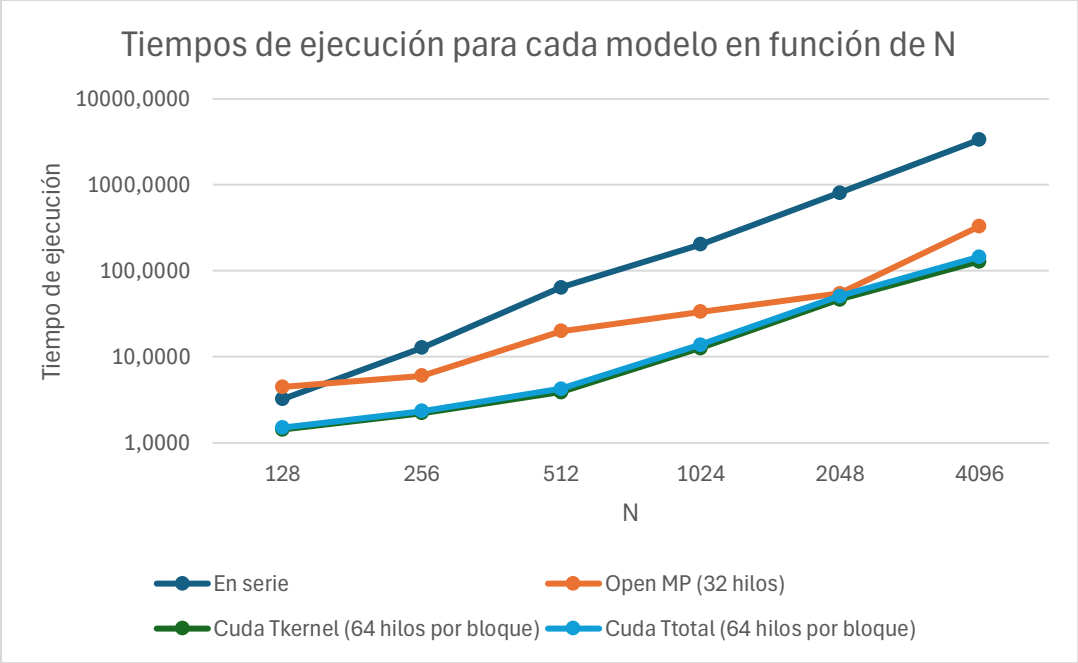
Al representarlos en las tablas y gráficas siguientes, observamos que, el caso en serie es el menos eficiente (excepto para valores pequeños, que lo es Open MP). Por lo tanto, para la ejecución de este programa compensa realizar cualquier tipo de paralelización, especialmente para valores grandes de N. Si bien Open MP mejora el rendimiento del programa en serie, como podemos observar, es mucho más eficiente utilizar la GPU, ya que, aun teniendo en cuenta el tiempo que se pierde realizando las copias de memoria, la versión Cuda es la más eficiente para todos los tamaños de N (al menos con esta configuración del kernel).

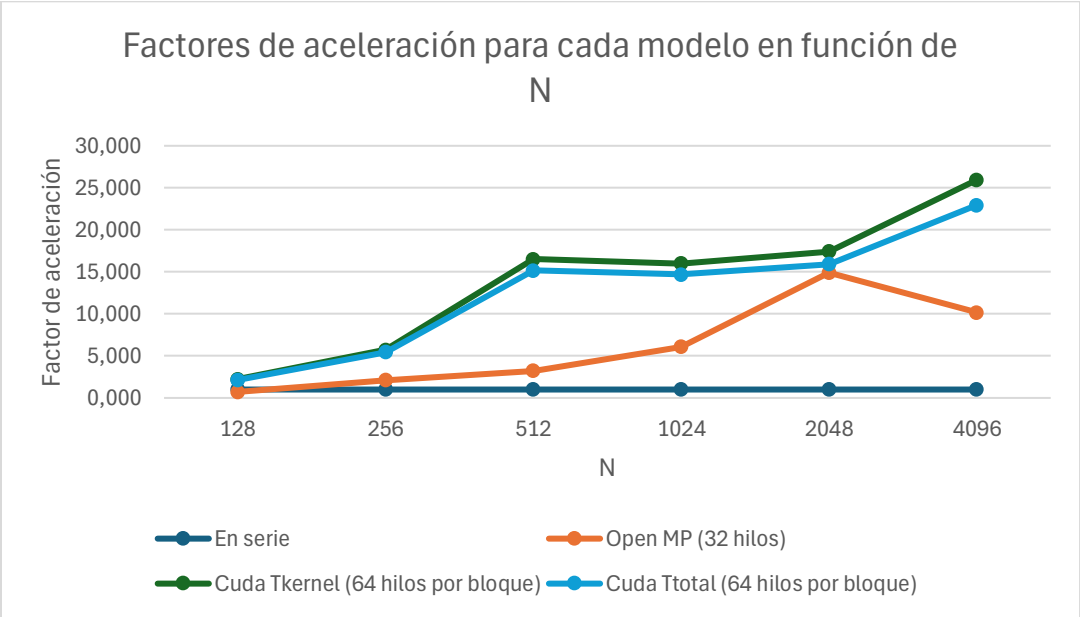
Por lo tanto si tendríamos que elegir una manera concreta de ejecutar el programa escogeríamos Cuda, ya que tiene un rendimiento notablemente mejor a las otras opciones planteadas.

TIEMPO DE EJECUCIÓN (ms)				
N	En serie	Open MP (32 hilos)	Cuda Tkernel (64 hilos por bloque)	Cuda Ttotal (64 hilos por bloque)
128	3,2193	4,4786	1,4308	1,5032
256	12,7166	6,0323	2,2229	2,3429
512	64,4296	19,9379	3,8983	4,2486
1024	202,5349	33,2983	12,6640	13,7960
2048	812,7538	54,5507	46,6134	51,0147

4096	3346,8291	328,6763	129,0270	145,8942

FACTOR DE ACELERACIÓN				
N	En serie	Open MP (32 hilos)	Cuda Tkernel (64 hilos por bloque)	Cuda Ttotal (64 hilos por bloque)
128	1,000	0,719	2,250	2,142
256	1,000	2,108	5,721	5,428
512	1,000	3,232	16,528	15,165
1024	1,000	6,082	15,993	14,681
2048	1,000	14,899	17,436	15,932
4096	1,000	10,183	25,939	22,940





Conclusiones

En el caso de estos programas, la versión en serie claramente tiene un mal rendimiento, ya que para valores grandes de N su tiempo de ejecución mejora notablemente. Si bien al paralelizarlo con Open MP los tiempos de ejecución mejoran, la mejor opción en ambos casos es paralelizarlo con Cuda, utilizando la GPU, ya que a pesar de que se pierde algo de tiempo al realizar las copias de memoria entre la CPU y la GPU, este tiempo es insignificante en relación con el tiempo que reducimos al ejecutar la función en el kernel. Por lo tanto, para ambos programas elegiríamos la versión del programa con la función paralelizable en el kernel por ser la que mejor rendimiento tiene.